

第8讲 存储器管理(1/3)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

存储器管理概述

02

单一连续区存储管理

02

分区存储管理

单一存储器的问题

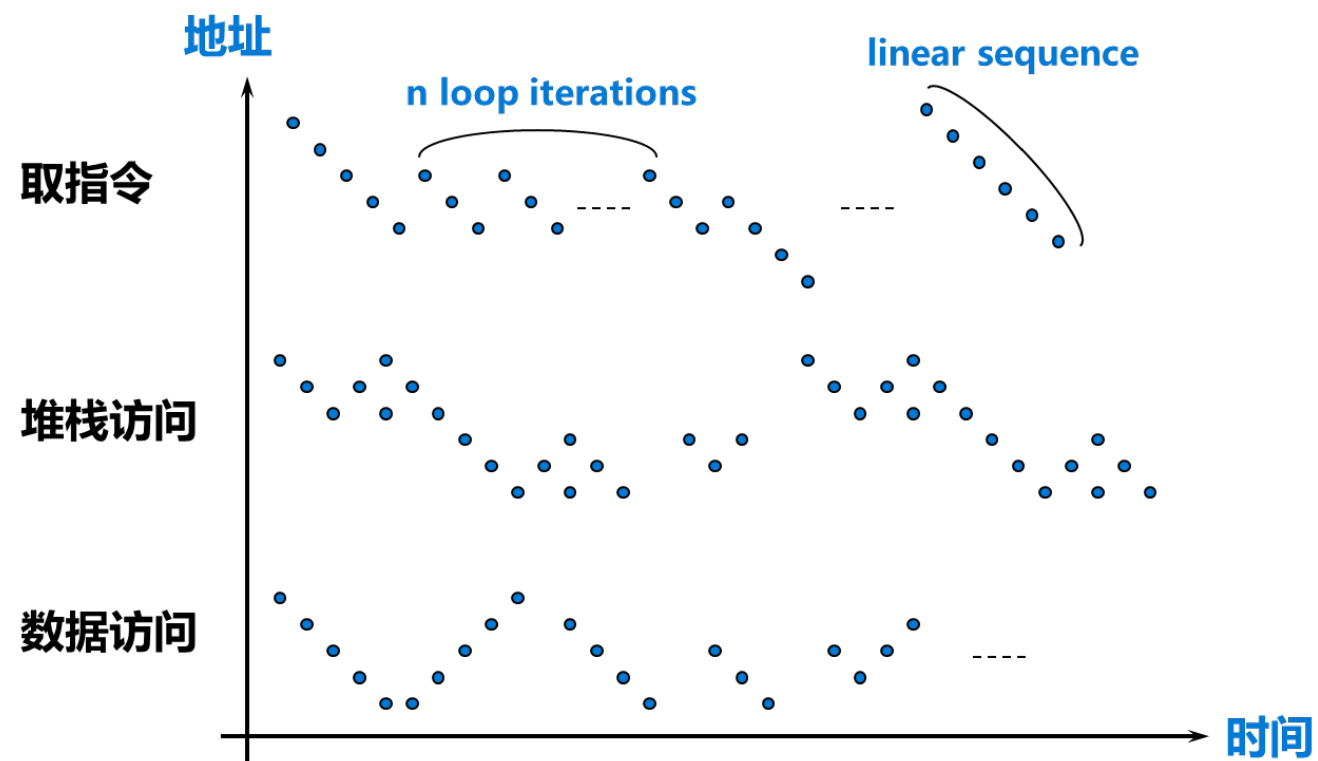
- 现代处理器能以很高的速度执行指令，因此处理器必须连接一个容量大、速度快的存储器
 - 如果存储器容量太小，就不能装载足够的程序以保持处理器全力处理
 - 如果太慢，就不能像处理器执行指令那样快地提供指令和数据
 - 成本也是一个重要的因素
- 不幸的是，设计一个足够大、足够快又足够便宜的单一存储器是不可能的

存储器访问的局部性原理

- 处理器访问存储器时，无论取指令还是取数据，所访问的存储单元都趋向于聚集在一个较小的连续单元区域中——**存储器访问的局部性原理**
 - **时间上的局部性**——最近的将来要用到的信息很可能就是现在正在使用的信息。主要由循环造成
 - **空间上的局部性**——最近的将来要用到的信息很可能与现在正在使用的信息在空间上是邻近的。主要由顺序执行和数据的聚集存放造成

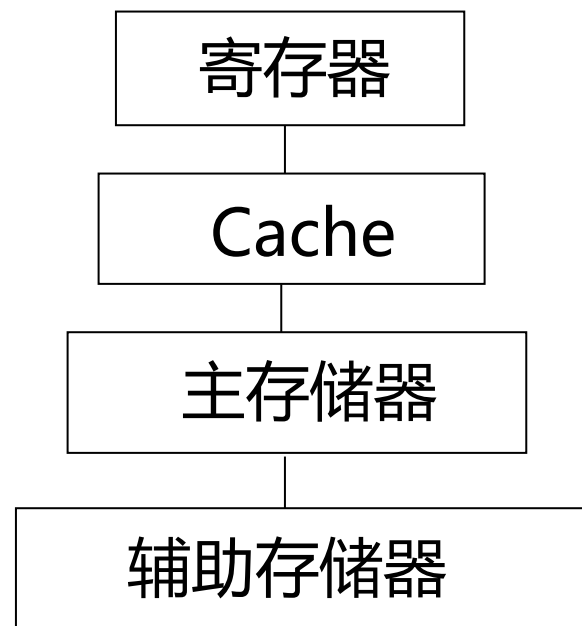
存储器访问的局部性原理

■典型的存储器访问模式



存储系统的层次结构

- 存储器的层次结构是依靠存储器访问的局部性实现的
- 现代计算机大多采用寄存器-高速缓存-主存储器(内存)-辅助存储器(外存)结构

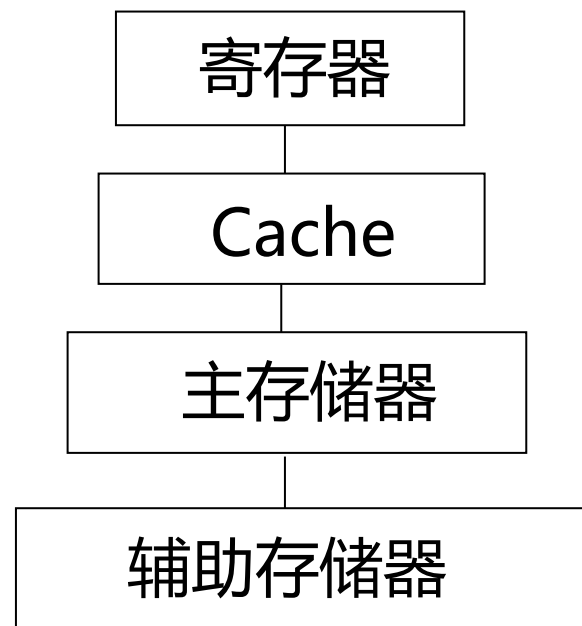


存储系统的层次结构

- 目前正在使用的指令或数据或者将要用到指令或数据应该在高速缓存中
- 过一段时间才会用到的指令或数据应该在主存储器中
- 暂时用不到的程序或数据应该在磁盘中
- 如果大部分访问都是对高速缓存进行的，那么我们就不会感觉到主存储器和磁盘的速度慢，好象我们有一个又大有快的存储器一样

存储系统的层次结构

- 高速缓存：SRAM
 - Data Cache、Code Cache
 - TLB(Translation Lookaside Buffer)
- 内存：DRAM、SDRAM等
- 外存：软盘、硬盘、光盘、磁带等



存储管理的功能

- 操作系统存储管理的功能针对的是内存
 - 外存属于设备(存储设备)
 - Cache对于软件不可见
- 内存管理的功能
 - 内存分配和回收
 - 地址变换与重定位
 - 内存保护
 - 内存扩充

内存分配和回收

- 完成内存的分配和回收工作，设计者需要选择和确定分配和回收策略以及相应的数据结构
 - 调入策略
 - 放置策略
 - 置换策略
 - 分配结构

内存分配和回收

■调入策略

- 用户程序在何时调入内存的策略

- 有[请调\(demand-paging\)](#)和[预调\(pre-paging\)](#)两种方法

■放置策略

- 用户程序调入内存时，确定将其放置在何处的策略

内存分配和回收

■ 置换策略

- 当需要将某个用户程序调入内存而内存空间又不够时，就要确定哪个或哪些程序可以从内存中移走

■ 分配结构

- 是用来登记内存使用情况的数据结构
- 如空闲分区表、空闲分区队列等

内存分配和回收的时机

- ① 进程的开始和结束
- ② 进程运行的过程中，它所占用的内存也可能发生变化，如栈的变化
- ③ 进程映像在内存和外存之间传递。由于内存有限，系统中不可能容纳所有进程，有些进程的映像可以存放在外存，当要运行这些进程时，必须把它们调入内存
- ④ 系统为了充分利用内存空间，有时可能对内存空间进行调整

地址变换与重定位

- 内存的每个存储单元都有一个编号，这种编号称为物理地址(或绝对地址，实地址)
- 物理地址的集合称为物理地址空间
- 要求用户用物理地址编程是非常困难的，尤其是在多道程序设计的环境中
- 程序从源代码到可执行文件，经过名空间——→相对地址空间——→逻辑地址空间的转换

地址变换与重定位

■ 名空间

- 程序员在程序中使用标识符命名变量、函数、类、对象等程序实体，并使用标识符在程序中用于引用和操作这些实体
- 程序中定义的标识符的集合称为程序的名空间
- 由程序员自定义
- 没有地址的概念

```
#include <stdio.h>

int main() {
    int num1 = 5;
    int num2 = 10;
    int sum = num1 + num2;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

地址变换与重定位

■相对地址空间

- 每个源程序在编译后，目标模块都以0为基地址顺序编址

```
#include <stdio.h>
```

```
int main() {  
    int num1 = 5;  
    int num2 = 10;  
    int sum = num1 + num2;  
    printf("The sum is: %d\n", sum);  
    return 0;  
}
```

File Type: COFF OBJECT

```
main:  
00000000: 55                push     ebp  
00000001: 8B EC            mov     ebp,esp  
00000003: 83 EC 0C        sub     esp,0Ch  
00000006: C7 45 FC 05 00 00 mov     dword ptr [ebp-4],5  
0000000D: C7 45 F8 0A 00 00 mov     dword ptr [ebp-8],0Ah  
00000014: 8B 45 FC        mov     eax,dword ptr [ebp-4]  
00000017: 03 45 F8        add     eax,dword ptr [ebp-8]  
0000001A: 89 45 F4        mov     dword ptr [ebp-0Ch],eax  
0000001D: 8B 4D F4        mov     ecx,dword ptr [ebp-0Ch]  
00000020: 51              push     ecx  
00000021: 68 00 00 00 00  push     offset $SG9728  
00000026: E8 00 00 00 00  call     _printf  
0000002B: 83 C4 08        add     esp,8  
0000002E: 33 C0           xor     eax,eax  
00000030: 8B E5          mov     esp,ebp  
00000032: 5D             pop     ebp  
00000033: C3             ret
```

相对地址

地址变换与重定位

■逻辑地址空间

- 由链接器生成，通过链接器的处理在标识符和逻辑地址之间建立映射关系
- 逻辑地址（或相对地址、虚拟地址）的集合称为逻辑地址空间
- 其首地址为0，其余指令中的地址都相对于首地址来编址
- 不能用逻辑地址在内存中读取信息

地址变换与重定位

逻辑地址空间

```
#include <stdio.h>

int main() {
    int num1 = 5;
    int num2 = 10;
    int sum = num1 + num2;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

Dump of file name.exe

File Type: EXECUTABLE IMAGE

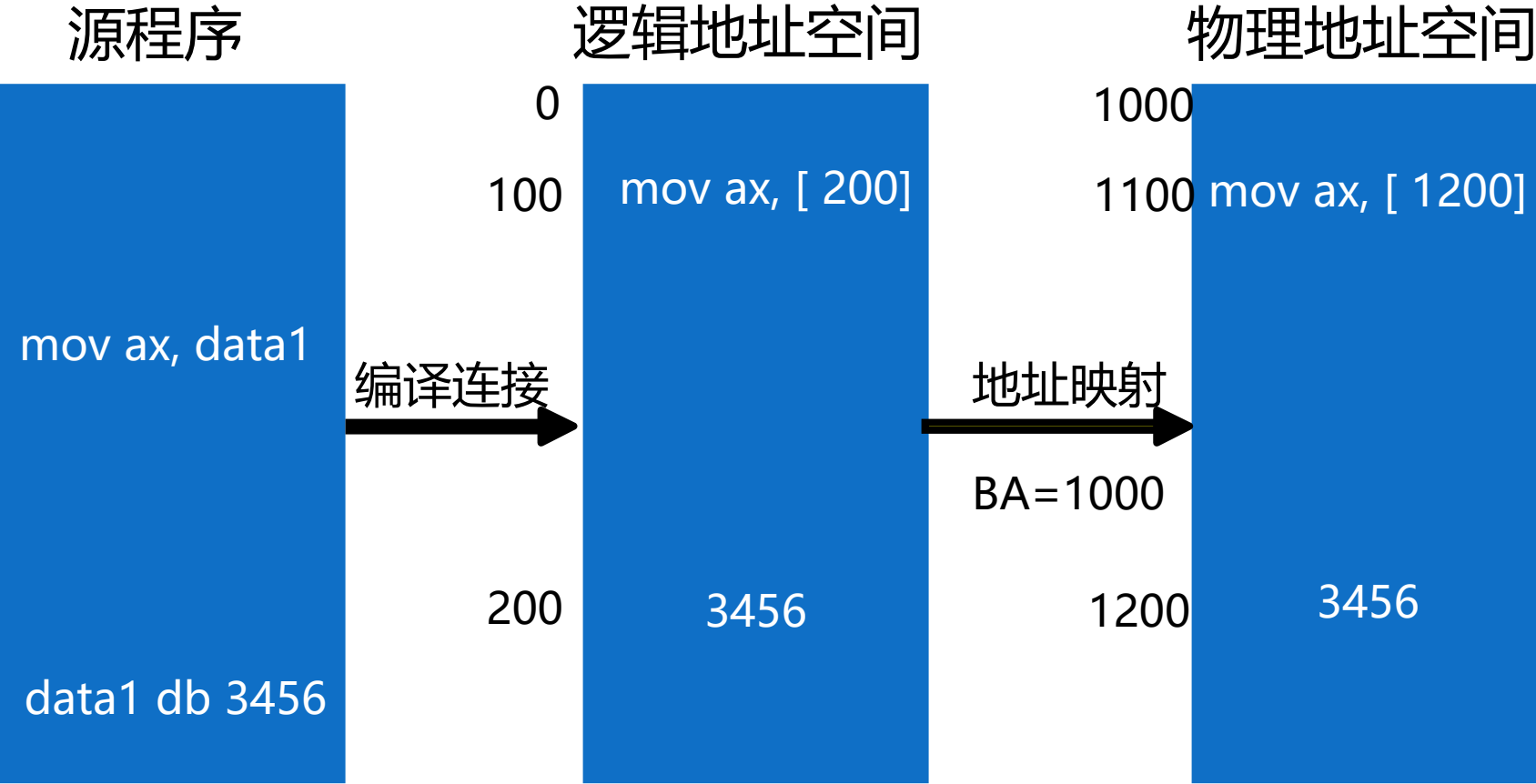
00401000:	55	push	ebp
00401001:	8B EC	mov	ebp, esp
00401003:	83 EC 0C	sub	esp, 0Ch
00401006:	C7 45 FC 05 00 00	mov	dword ptr [ebp-4], 5
	00		
0040100D:	C7 45 F8 0A 00 00	mov	dword ptr [ebp-8], 0Ah
	00		
00401014:	8B 45 FC	mov	eax, dword ptr [ebp-4]
00401017:	03 45 F8	add	eax, dword ptr [ebp-8]
0040101A:	89 45 F4	mov	dword ptr [ebp-0Ch], eax
0040101D:	8B 4D F4	mov	ecx, dword ptr [ebp-0Ch]
00401020:	51	push	ecx
00401021:	68 00 B0 41 00	push	41B000h
00401026:	E8 55 00 00 00	call	00401080
0040102B:	83 C4 08	add	esp, 8
0040102E:	33 C0	xor	eax, eax
00401030:	8B E5	mov	esp, ebp
00401032:	5D	pop	ebp
00401033:	C3	ret	

逻辑地址

地址变换与重定位

- 将用户程序中的逻辑地址转换为运行时由机器直接寻址的物理地址的过程称为**地址变换**，也称为**地址重定位**
- 当程序装入内存时，操作系统要为该程序分配一个合适的内存空间，由于程序的逻辑地址与分配到内存物理地址不一致，而CPU访问内存时，是按物理地址进行的，所以要进行地址转换

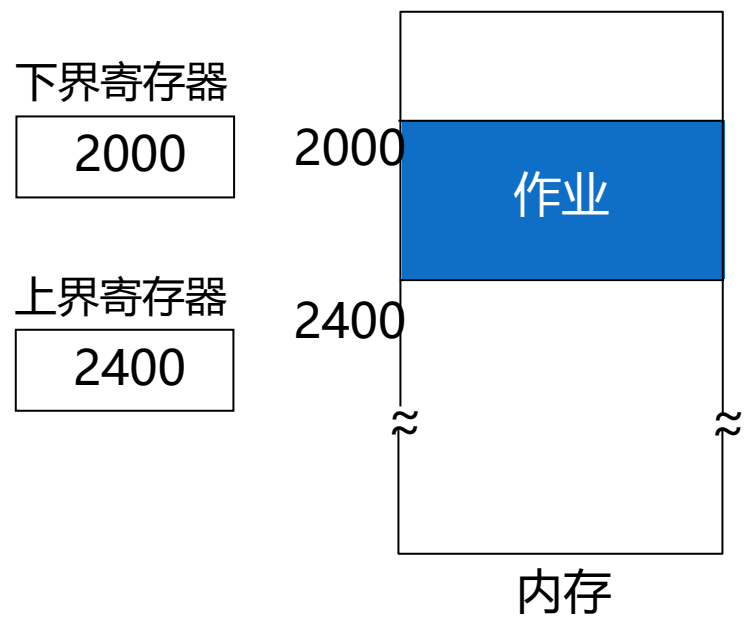
地址变换与重定位



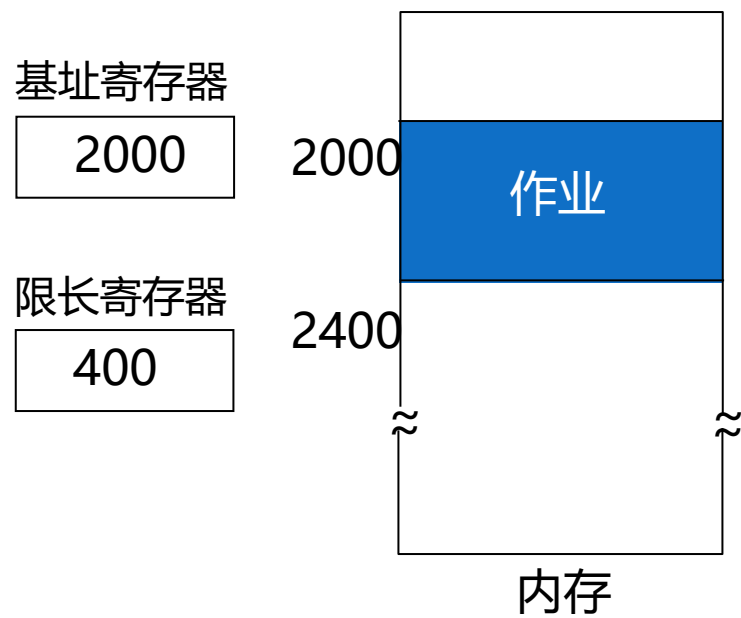
内存保护

- 保证在内存中的多道程序只能在给定的内存区域内活动，互不产生干扰
- 在CPU中设置一对**下限寄存器**和**上限寄存器**存放用户作业在内存中的下限和上限地址
- 也可将一个寄存器作为**基址寄存器**，另一寄存器作为**限长寄存器**(指示存储区长度)
- 每当CPU要访问主存，硬件自动将被访问的主存地址与界限寄存器的内容进行比较，以判断是否越界
- 如果未越界，则按此地址访问主存，否则将产生程序中断——越界中断(内存保护中断)

内存保护



上、下界寄存器保护



基址、限长寄存器保护

内存保护——共享问题

- 为了在多道程序环境下充分利用内存空间，应当允许多个进程并发执行同一段代码，或者相互协作的进程共享访问同一数据结构
- 共享是内存保护功能的增强——保护机制在得到许可的情况下，应该允许多个进程访问内存的同一部分

内存扩充

- 实际内存的容量不应该限制用户程序的大小，操作系统的存储管理的功能应该允许运行任意大小的程序
- 由应用程序控制
 - 覆盖技术
- 由OS控制
 - 交换技术——针对整个进程空间
 - 虚拟存储器技术——针对部分进程空间

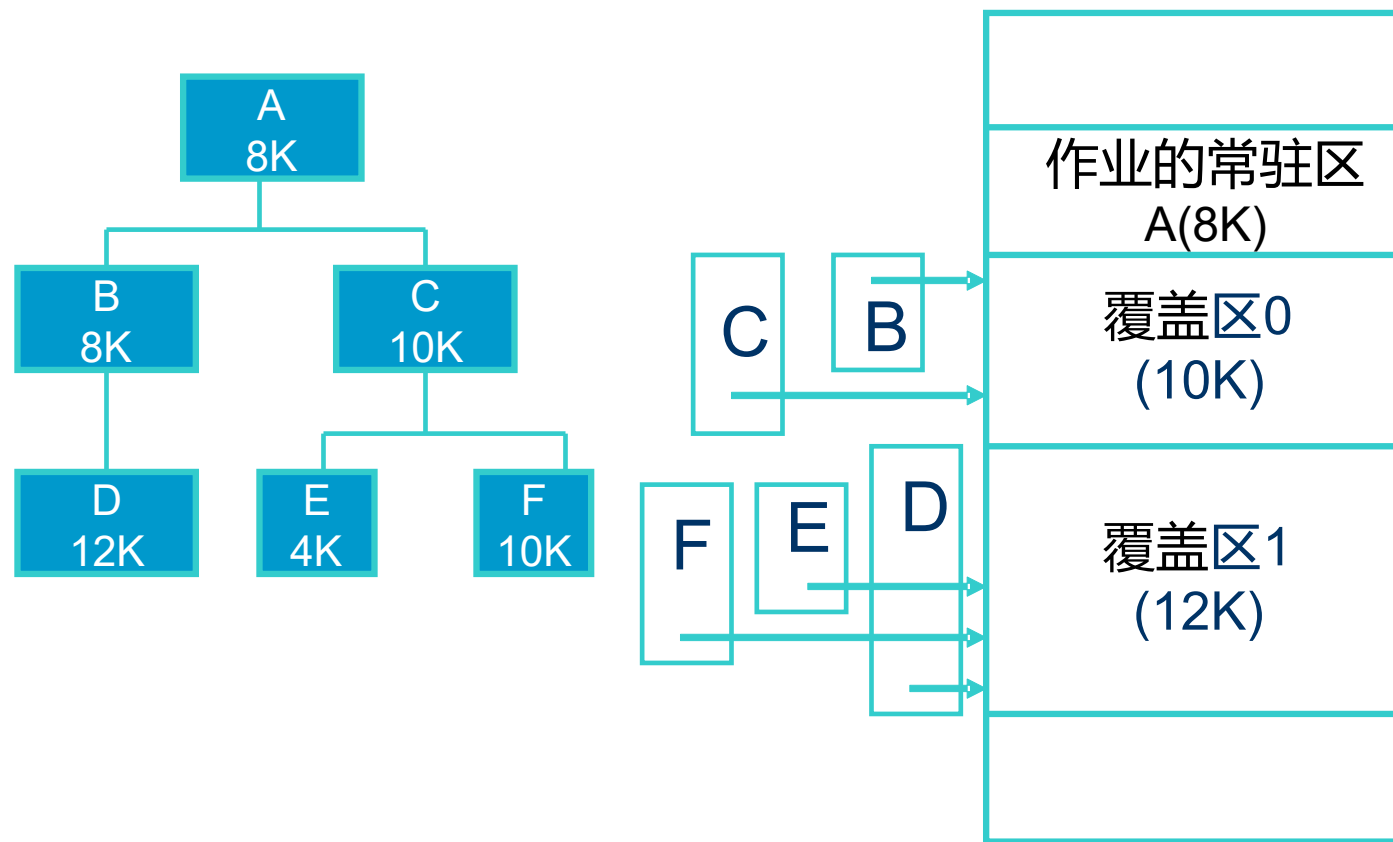
内存扩充——覆盖技术

- 多年以前，人们经常遇到程序太大，以至于内存容纳不下的问题
- 将程序分割成许多片断，称为覆盖(overlay)。覆盖块0首先运行，当它结束时，调入另一个覆盖块
- 覆盖块存放在磁盘上，在需要时通过调用操作系统服务动态地换入换出
- 虽然由操作系统实际完成覆盖块的换入换出，但是程序员必须把程序分割成许多片断

内存扩充——覆盖技术

- 把程序划分为若干个功能上相对独立的程序段，按照其自身的逻辑结构将那些不会同时执行的程序段共享同一块内存区域
- 程序段先保存在磁盘上，当有关程序段的前一部分执行结束，把后续程序段调入内存，覆盖前面的程序段(内存“扩大”了)

内存扩充——覆盖技术



内存扩充——交换技术

■交换技术(swapping)

- 当内存空间紧张时，系统将内存中阻塞进程暂时换出到外存，把外存中具备运行条件的进程换入到内存，占据前者所占用的区域，这种技术是进程在内存与外存之间的动态调度（中级调度）
- 多用于分时系统中

内存扩充——交换技术

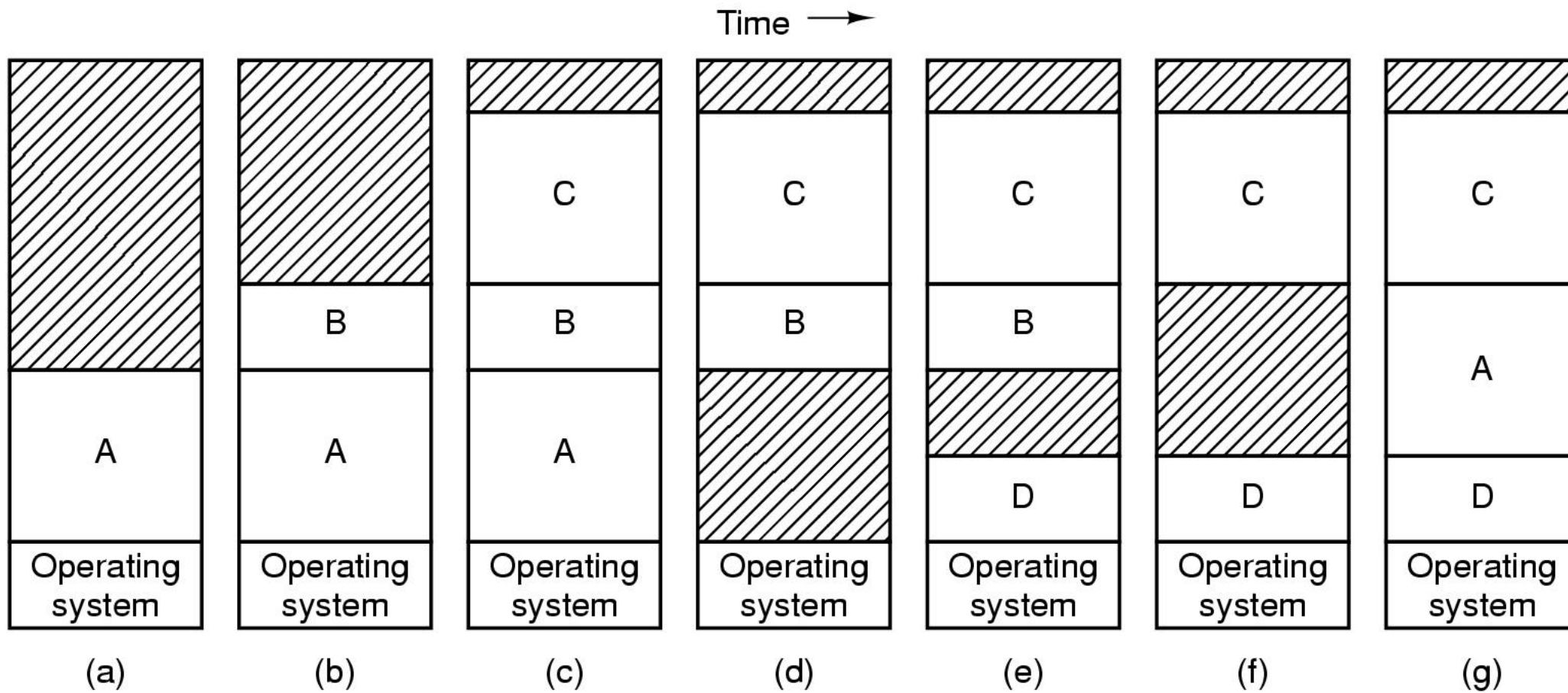
■何时需发生交换

- 只要不用就换出(或很少再用)
- 只在内存空间不够或有不够的危险时换出

■需要一个磁盘交换区

- 必须足够大以存放用户程序的内存映像的拷贝
- 交换区比文件区侧重于交换速度，因此，交换区一般采用连续分配

内存扩充——交换技术

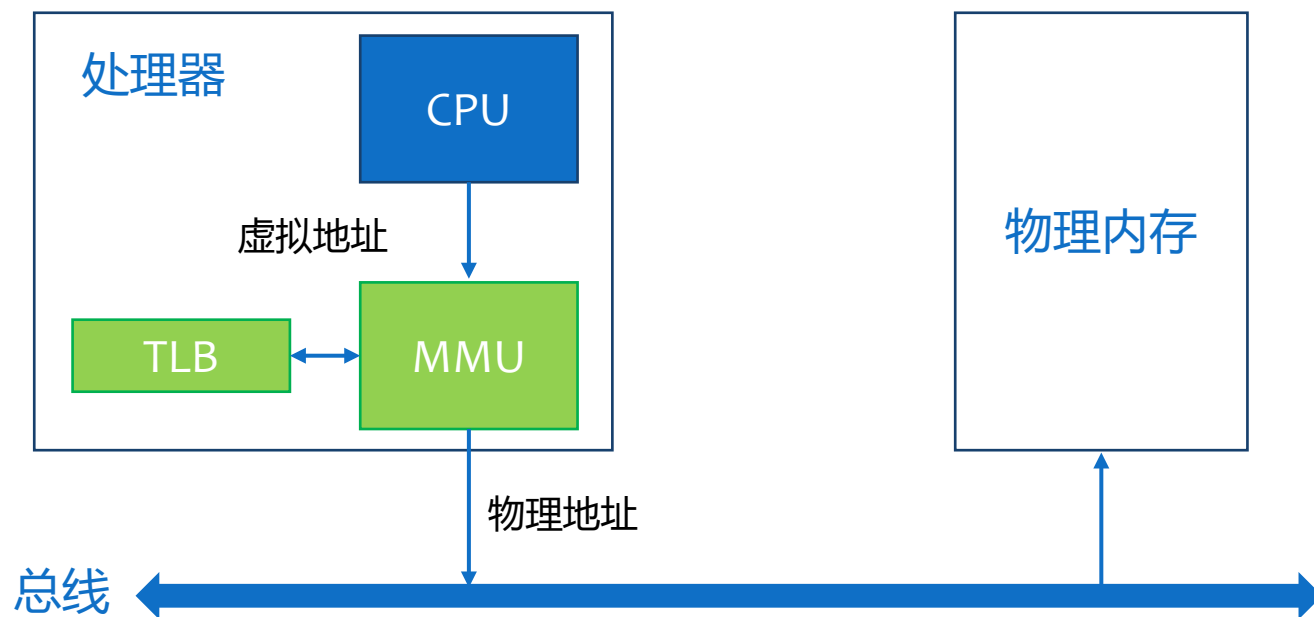


内存扩充——虚拟存储器

- 将程序正在使用的部分内容放在内存，而暂时不用的部分放在外存，在需要时由系统调入内存，并将不需要(或暂不需要)的部分调出内存
- 由于存储器访问的局部性，程序在执行时，在一段时间内一般仅使用它的程序的一部分，所以程序仅有部分装入内存完全能够正确执行
- 使程序员在编程时不受内存的结构和容量的限制，这种存储器称为**虚拟存储器(virtual memory)**
- 由虚拟存储器构成的存储空间称为**虚地址空间**

内存扩充——虚拟存储器

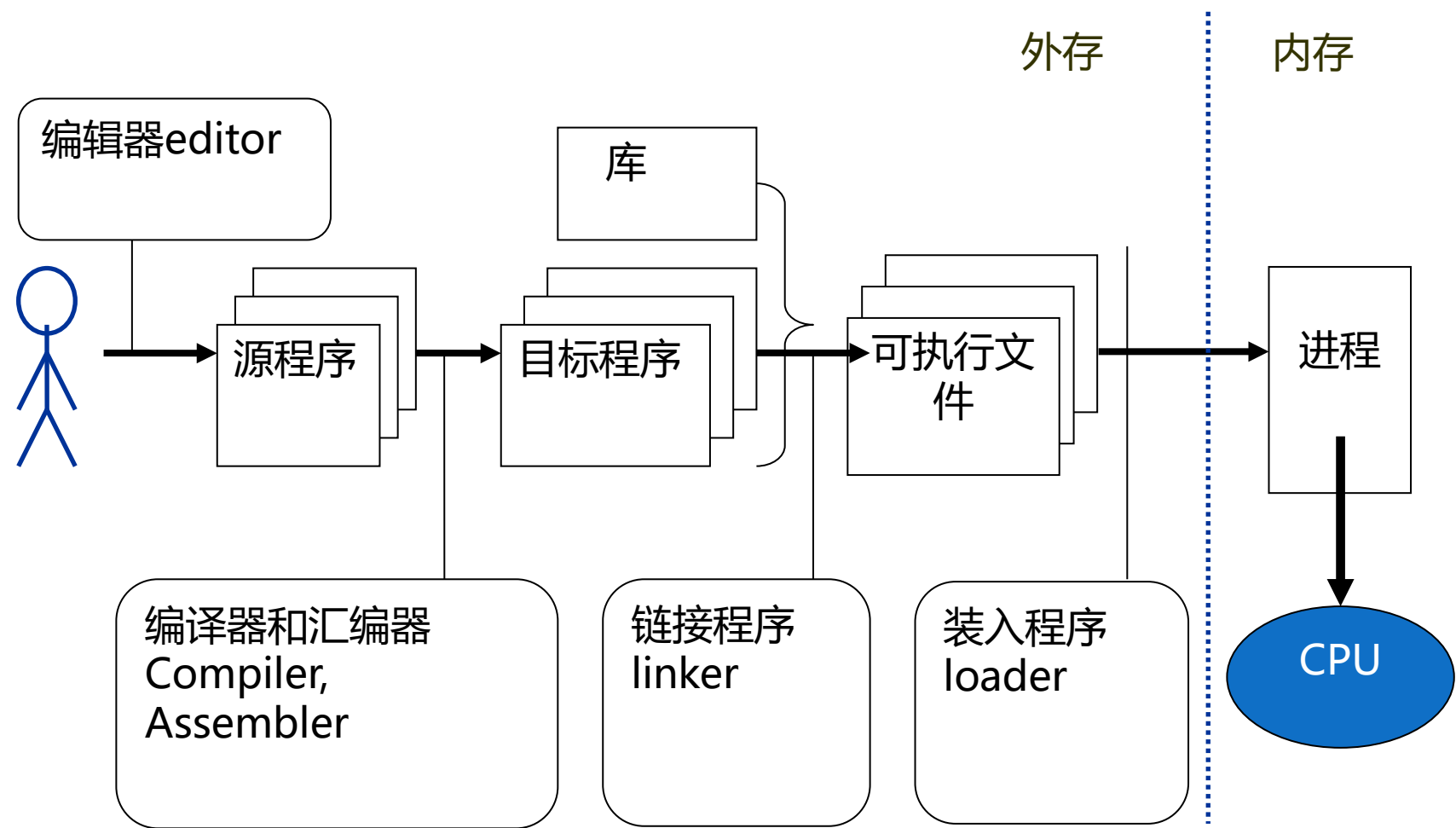
- 由OS结合相关硬件来完成虚拟地址到物理地址的转换工作



程序的链接与装入

- 用户源程序要执行，通常要经过以下几个步骤：
 1. 编译——由编译程序将用户源代码编译成若干个目标模块
 2. 链接——由链接程序将编译后形成的目标模块以及它们所需要的库函数，链接在一起，结合到同一个逻辑地址空间中，形成一个装入模块(可执行程序)
 3. 装入——由装入程序将装入模块装入主存的过程(加载)

程序的链接与装入



程序链接

- 经过汇编或编译生成的所有可重定位的目标模块都采用首地址为0的相对地址，模块中其余的指令地址和数据地址都相对于首地址而编
- 链接程序的功能是将这组目标模块以及他所需要的库函数装配成一个完整装入模块（可执行文件）

程序链接方式

- 静态链接——事先进行链接，形成可执行文件
 - 修改相对地址
 - 变换外部调用符号
- 装入时动态链接——边装入边链接
 - 便于修改和更新
 - 便于实现对目标模块的共享
- 运行时动态链接——需要时再链接
 - 节省内存空间

程序的装入

- 程序的装入就是把程序装入内存空间
- 程序的装入有方法
 - 绝对装入(absolute loading)
 - 可重定位装入(relocatable loading)
 - 动态装入(dynamic run-time loading)

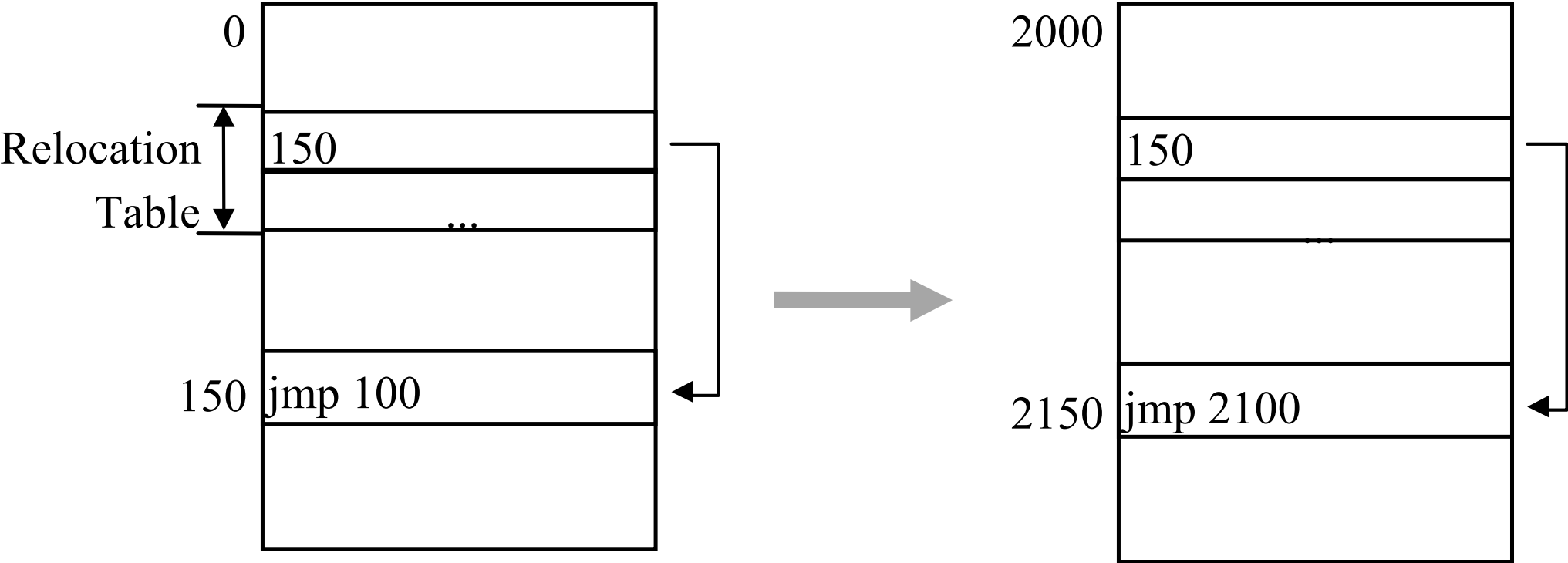
绝对装入

- 在可执行文件中记录内存地址，装入时直接定位在该内存地址(即文件中记录的地址)
 - 优点：装入过程简单
 - 缺点：过于依赖于硬件结构，不适于多道程序系统

可重定位装入

- 在可执行文件中，包含重定位表，其中列出各个需要重定位的地址单元和相对地址值。当用户程序被装入内存时，**一次性**实现逻辑地址到物理地址的转换，以后不再转换。即装入时根据所定位的内存地址去修改每个重定位地址项，添加相应偏移量
- 优点：
 - 不需硬件支持，可以装入有限多道程序
- 缺点：
 - 一次性全部装入
 - 一个程序通常需要占用连续的内存空间
 - 程序装入内存后运行时不能移动

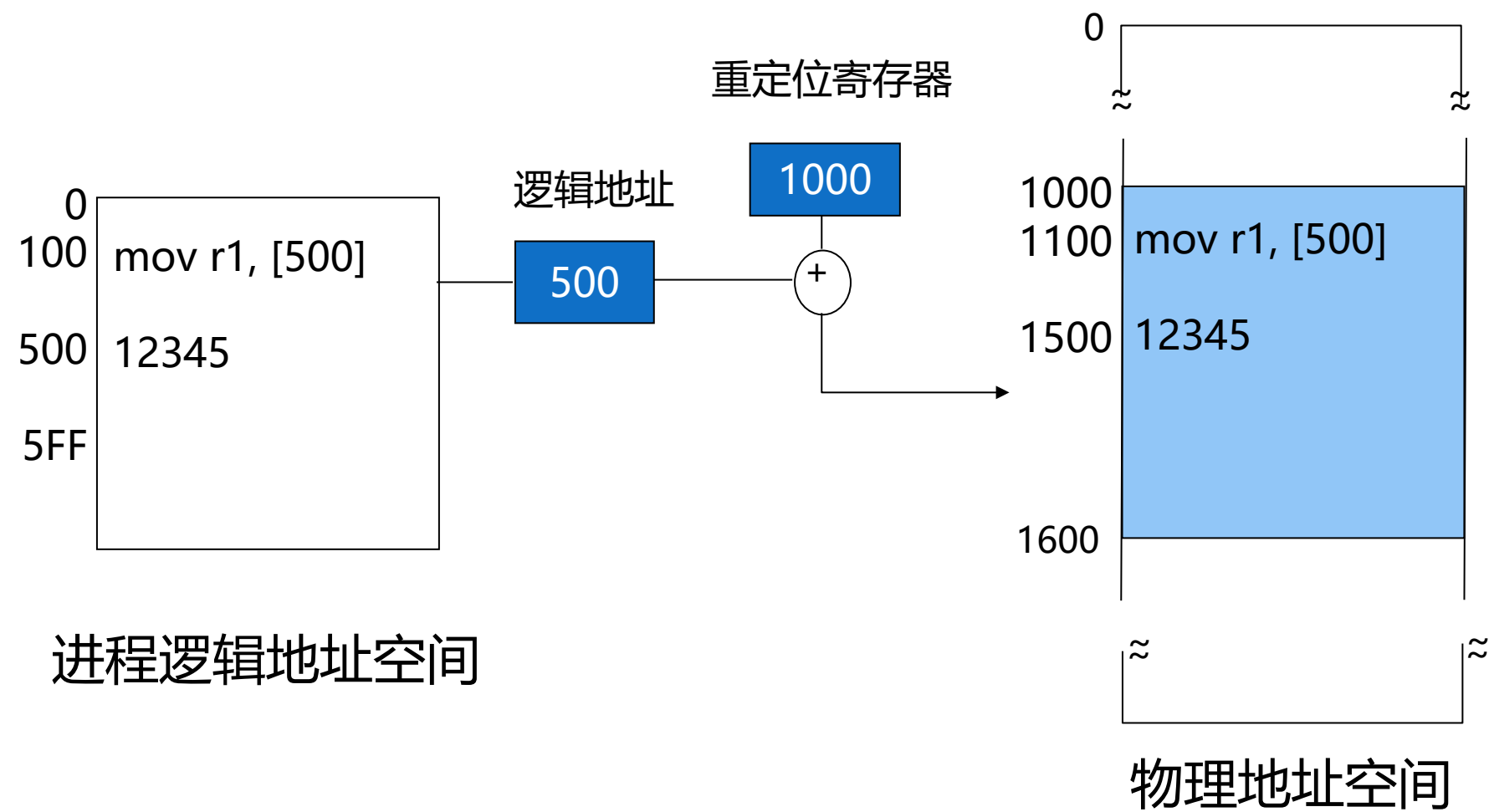
可重定位装入



动态装入

- 在可执行文件中记录虚拟地址/逻辑地址，在装入内存时，并不立即把装入模块中的虚拟地址转换为物理地址，而是把这种地址转换推迟到程序要真正执行时才进行。执行时通过硬件地址变换机构，完成虚拟地址到实际内存地址的变换
- 优点：
 - OS可以将一个程序分散存放于不连续的内存空间，可以移动程序，有利于实现共享
 - 能够支持程序执行中产生的地址引用，如指针变量(而不仅是生成可执行文件时的地址引用)
 - 它是虚拟存储的基础
- 缺点：需要硬件支持(通常是MMU)，OS实现较复杂

动态装入



如下程序代码局部性体现出存储器访问的

```
int sum= 0;  
for (int i=0; i<N; i++) {  
    sum += a[i];  
}
```

//a是一个整型数组

- ☐ A 空间局部性
- ☐ B 时间局部性
- ☒ C 空间局部性和时间局部性
- ☐ D 没有局部性

提交

下述关于覆盖技术的描述，正确的是

- ☒ A 覆盖块的最大长度受物理内存容量的限制
- ☐ B 覆盖块的划分由操作系统负责
- ☒ C 覆盖块具有相对独立性
- ☐ D 覆盖块可以相互交叉访问

提交

下述关于交换技术的描述，正确的是

- ☒ A 进程的大小受物理内存容量的限制
- ☒ B 交换技术调入/调出整个进程
- ☐ C 交换技术可以调入/调出进程的部分代码或数据
- ☐ D 何时交换由程序员决定

提交

大纲

01

存储器管理概述

02

单一连续区存储管理

02

分区存储管理

连续分配存储管理

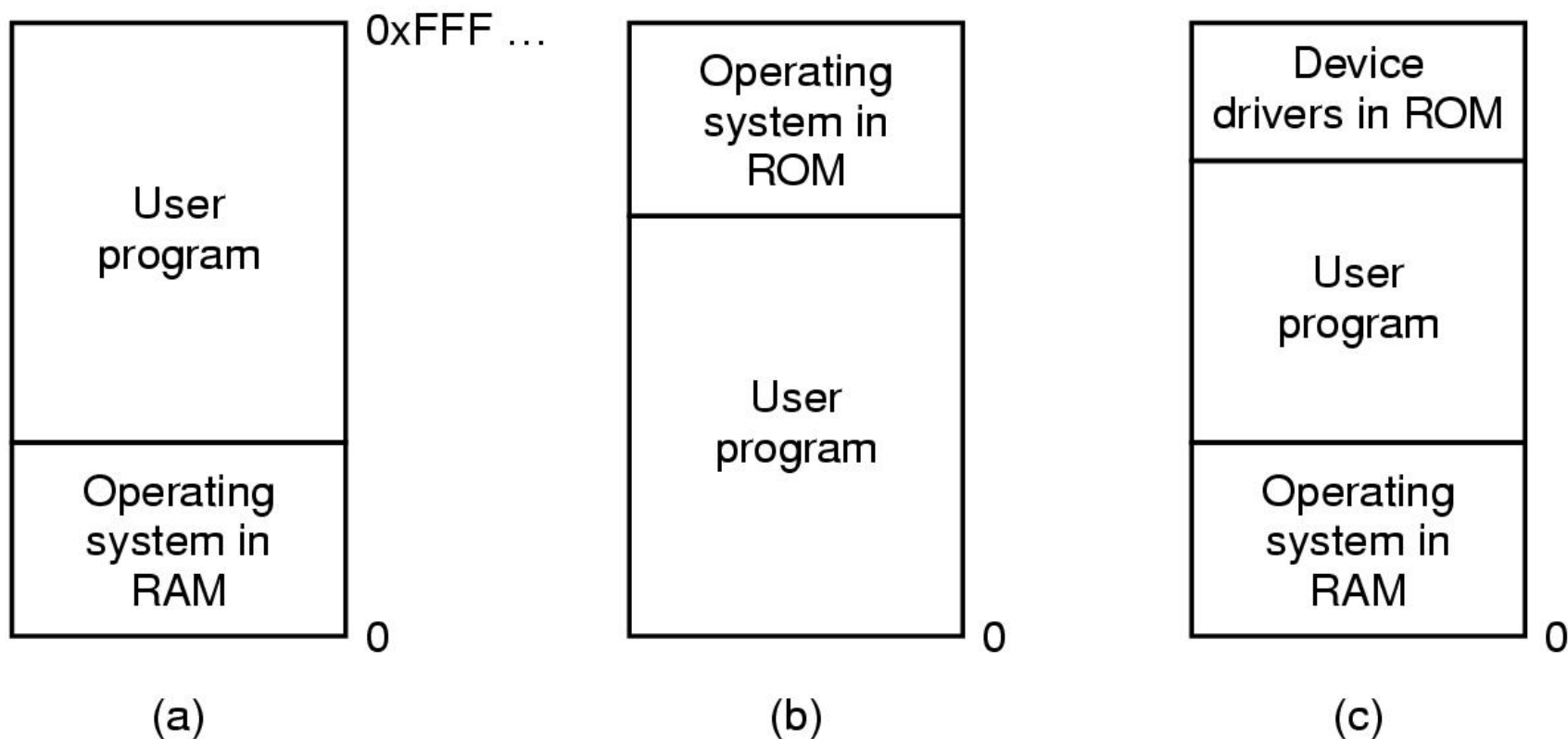
- 物理内存分配可以分为连续分配和非连续分配
- 连续分配，是指为一个用户程序分配一个连续的物理内存空间
- 连续分配方式进一步分为：单一连续区分配和分区分配

单一连续区存储管理

- 内存分为操作系统区和用户区。应用程序装入到用户区，可使用用户区全部空间
- 适用于单用户、单任务的操作系统
- 实例：MS-DOS，单片机系统

单一连续区存储管理

■单一连续区存储管理组织内存的方法



单一连续区存储管理

- 需要硬件保护机构，以确保用户程序不至于偶然或无意地干扰系统区中的信息
- 解决方法：使用界限寄存器，作业运行时，检查内存访问的地址，若不在界限寄存器所限定的范围内，则发生越界中断

单一连续区存储管理

- 优点：方法简单，对硬件要求低，易于实现
- 缺点：程序全部装入，很少使用的程序部分也占用内存，对要求内存空间少的程序，造成内存浪费

大纲

01

存储器管理概述

02

单一连续区存储管理

02

分区存储管理

分区存储管理

- 把内存分为一些大小相等或不等的分区(partition)，每个进程占用一个或几个分区。操作系统占用其中一个分区
- 适用于多道程序系统和分时系统

分区存储管理

- 分区存储管理可能存在内碎片和外碎片
 - 内碎片：占用分区之内未被利用的空间
 - 外碎片：占用分区之间难以利用的空闲分区(通常是小空闲分区)
- 分区存储管理有固定分区(fixed partitioning)和动态分区(dynamic partitioning)两种方法

固定分区

- 把内存划分为若干个固定大小的连续分区，每个分区的边界固定
 - 分区大小相等：适合于多个相同大小的程序并发执行
 - 分区大小不等：多个小分区、适量的中等分区、少量的大分区。根据程序的大小，分配当前空闲的、适当大小的分区
- 没有外碎片，可能有内碎片

固定分区

Operating System 8M
8M
8M
8M
8M

固定分区(相同大小)

Operating System 8M
2M
4M
6M
8M
8M
12M

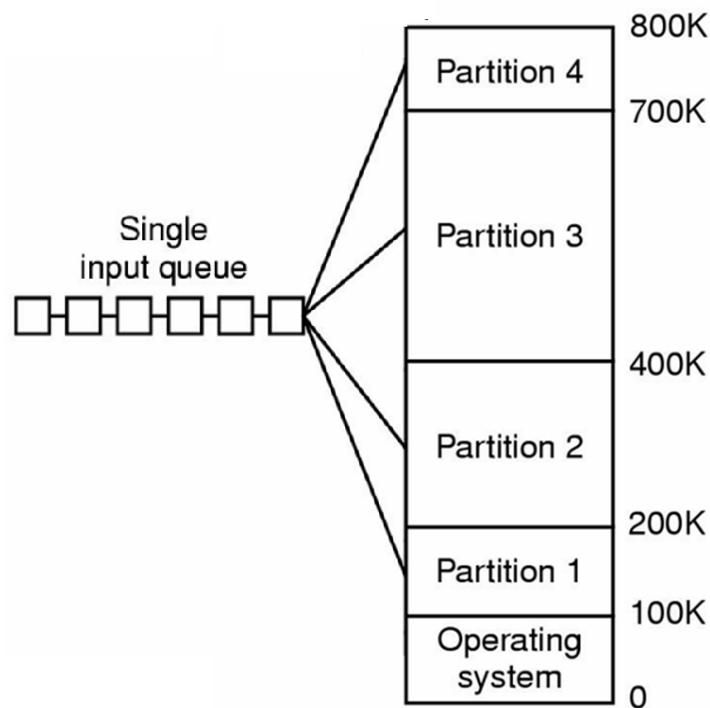
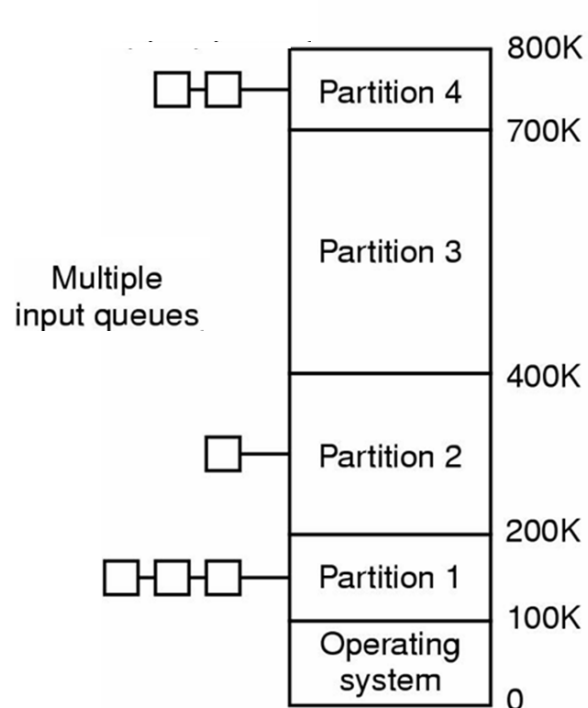
固定分区(多种大小)

分区大小相等的固定分区

- 由于每个分区大小相等，所以当有一个进程调入内存时，可以放入任何一个空闲分区——**不需要放置策略**
- 如果没有分区空闲，操作系统可以选择内存中一个处于非运行状态的进程，将其换出到外存——采用**交换技术**
- 当程序很大以致超过了分区的大小时，可以采用**覆盖技术**
- 当程序很小时，也要将整个分区分配给它，导致较大的内碎片——**最大的缺点**

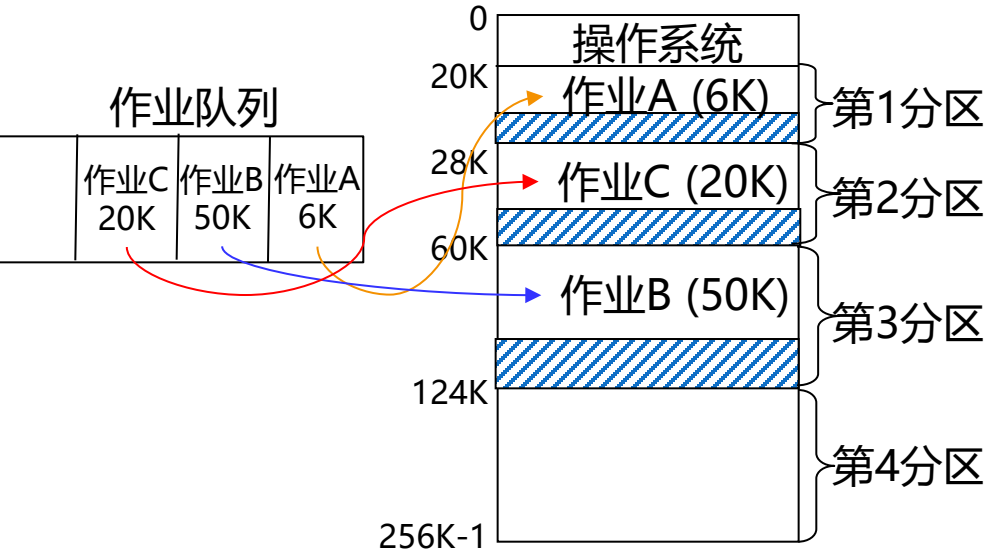
分区大小不等的固定分区

- **放置策略**：当一个作业到达时，将其放在能够容纳这个作业的最小分区中



固定分区

■ 为便于管理，系统要建立分区表，记录分区大小、起始地址和状态等信息



分区表

分区号	大小 (KB)	始址	状态
1	8	20	已分配
2	32	28	已分配
3	64	60	已分配
4	132	124	未分配

固定分区

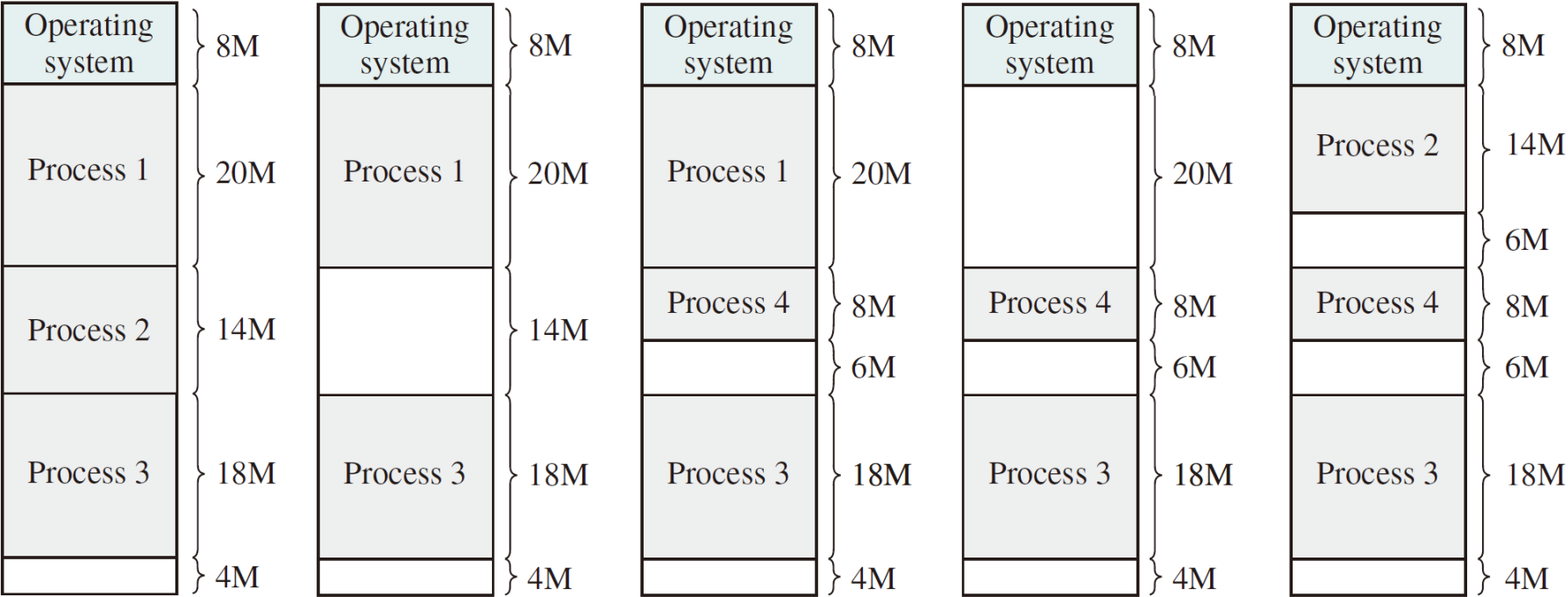
- 优点：易于实现，开销小
- 缺点：
 - 内碎片造成浪费
 - 分区总数固定，限制了并发执行的程序数目

动态分区

- 并不预先将内存事先划分成分区，当程序需要装入内存时系统从空闲的内存区中分配大小等于程序所需的内存空间
- 在进程执行过程中可以通过系统调用改变分区大小
- 分区的大小、数量和位置随着内存中进程的大小和数量而变化
- 没有内碎片，可能有外碎片

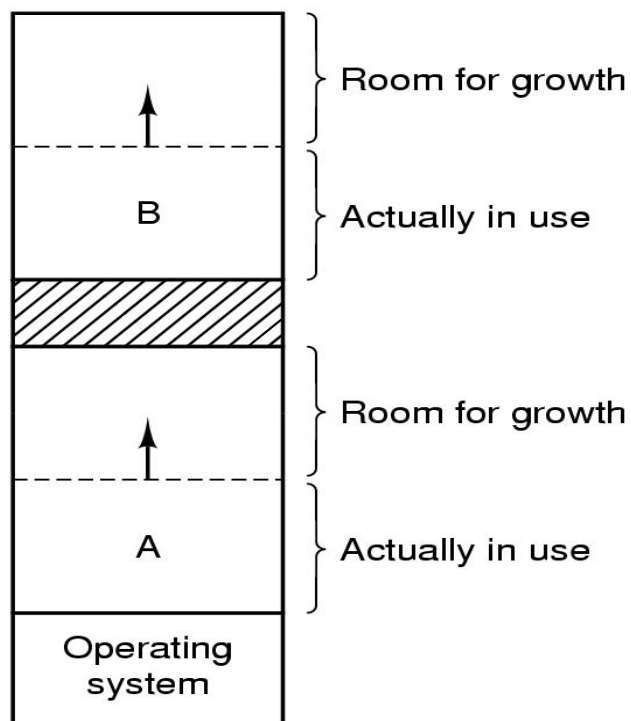
动态分区

■ 假设，进程1：20M，进程2：14M，进程3：18M，进程4：8M。动态分区变化过程

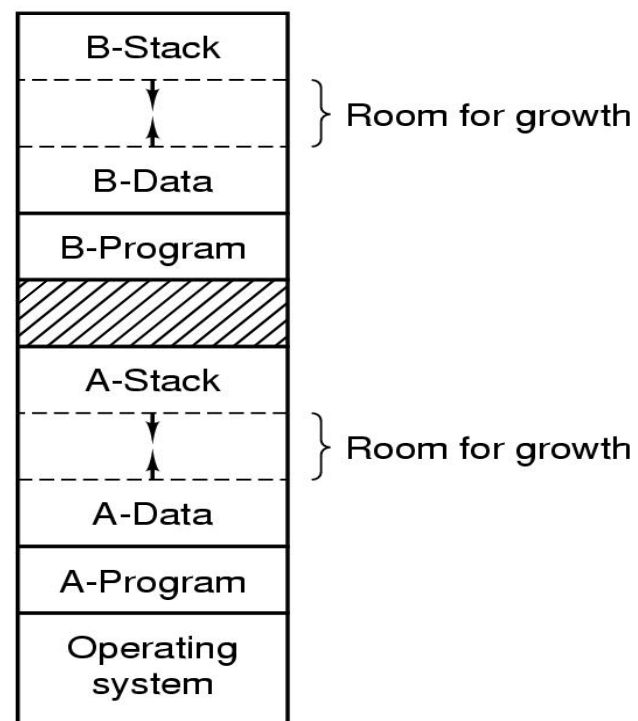


动态分区

■ 为可能增长的进程预留空间



(a)



(b)

动态分区

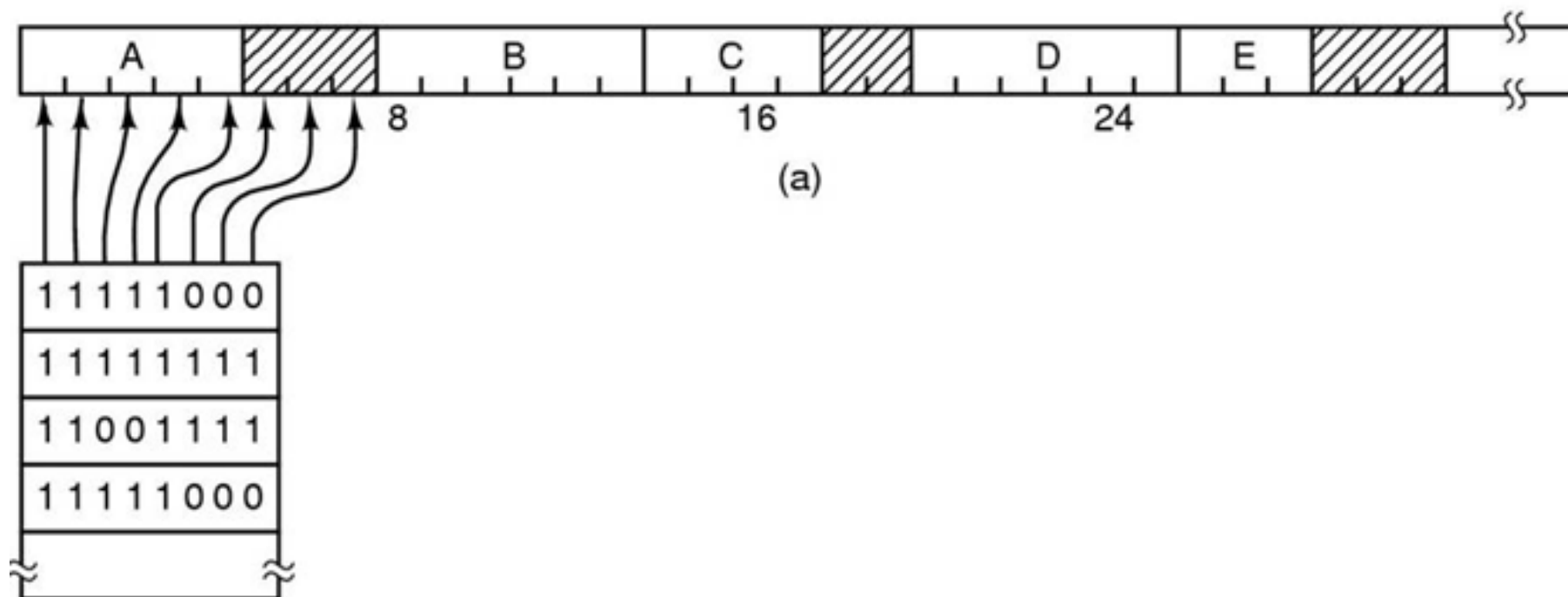
- 操作系统可以采用两种方法跟踪内存的使用情况
 - 位图
 - 空闲链表

使用位图的存储管理

- 内存划分成小的分配单位，一个分配单位可以小到几个字，大到几KB
- 每个分配单位对应位图中的一个位：
 - 1表示占用
 - 0表示空闲

使用位图的存储管理

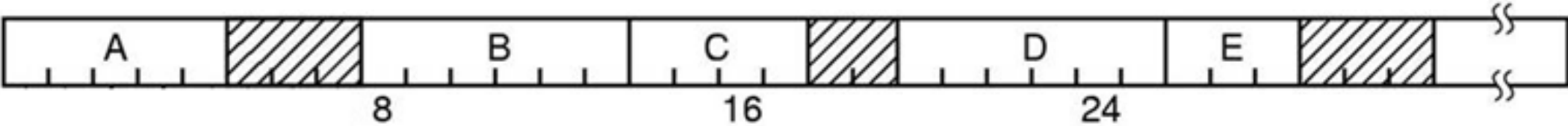
- 当一个占用 k 个分配单位的进程调入内存时，存储管理器必须搜索位图，找出 k 个连续的0



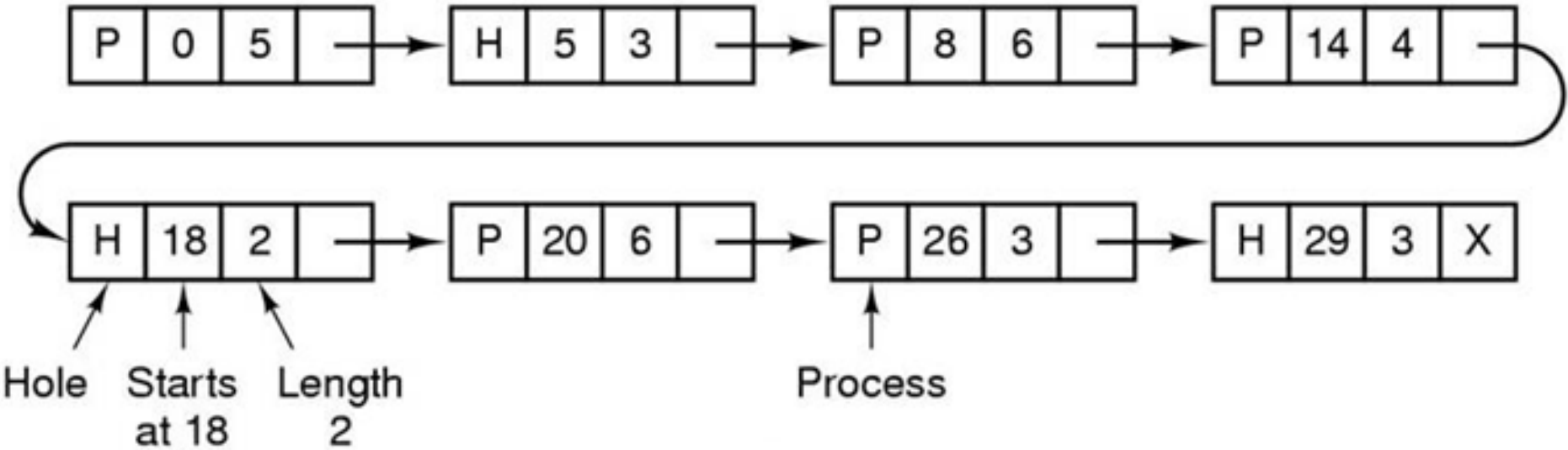
使用链表的存储管理

- 维护一个记录已分配内存分区和空闲内存分区的链表，链表中的每一项或者表示一个进程，或者表示两个进程间的一个空闲分区
- 链表中每个表项的内容：
 - 指示标志：空闲区(H)，进程(P)
 - 起始地址
 - 长度
 - 指向下一表项的指针

使用链表的存储管理

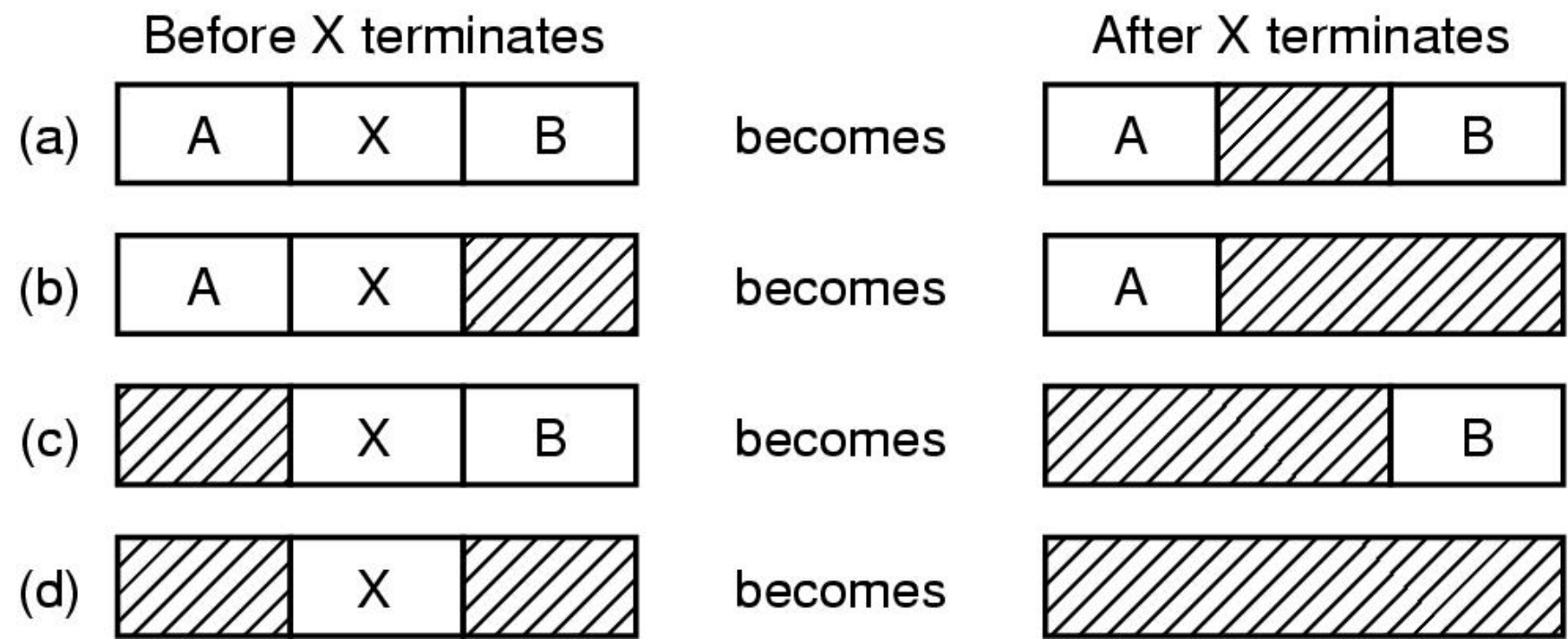


(a)



使用链表的存储管理

■ 当进程终止或被换出时链表的更新



分区分配算法

- 寻找某个空闲分区，其大小需大于或等于程序的要求
 - 首次适配法(first-fit)
 - 下次适配法(next-fit)
 - 最佳适配法(best-fit)
 - 最坏适配法(worst-fit)
- 若是大于要求，则将该分区分割成两个分区，其中一个分区为要求的大小并标记为“占用”，而另一个分区为余下部分并标记为“空闲”

首次适配法

- 按分区的先后次序，从头查找，找到符合要求的第一个分区
- 该算法的分配和释放的时间性能较好，较大的空闲分区可以被保留在内存高端
- 但随着低端分区不断划分而产生较多小分区，每次分配时查找时间开销会增大。同时，这小分区（外碎片）可能得不到利用，造成浪费

下次适配法

- 按分区的先后次序，从上次分配的分区起查找(到最后分区时再回到开头)，找到符合要求的第一个分区
- 该算法的分配和释放的时间性能较好，使空闲分区分布得更均匀，但较大的空闲分区不易保留

最佳适配法

- 按分区的先后次序，找到其大小与要求相差最小的空闲分区
- 外碎片较小，但随着运行时间的增长会积累较多小碎片。较大的空闲分区可以被保留

最坏适配法

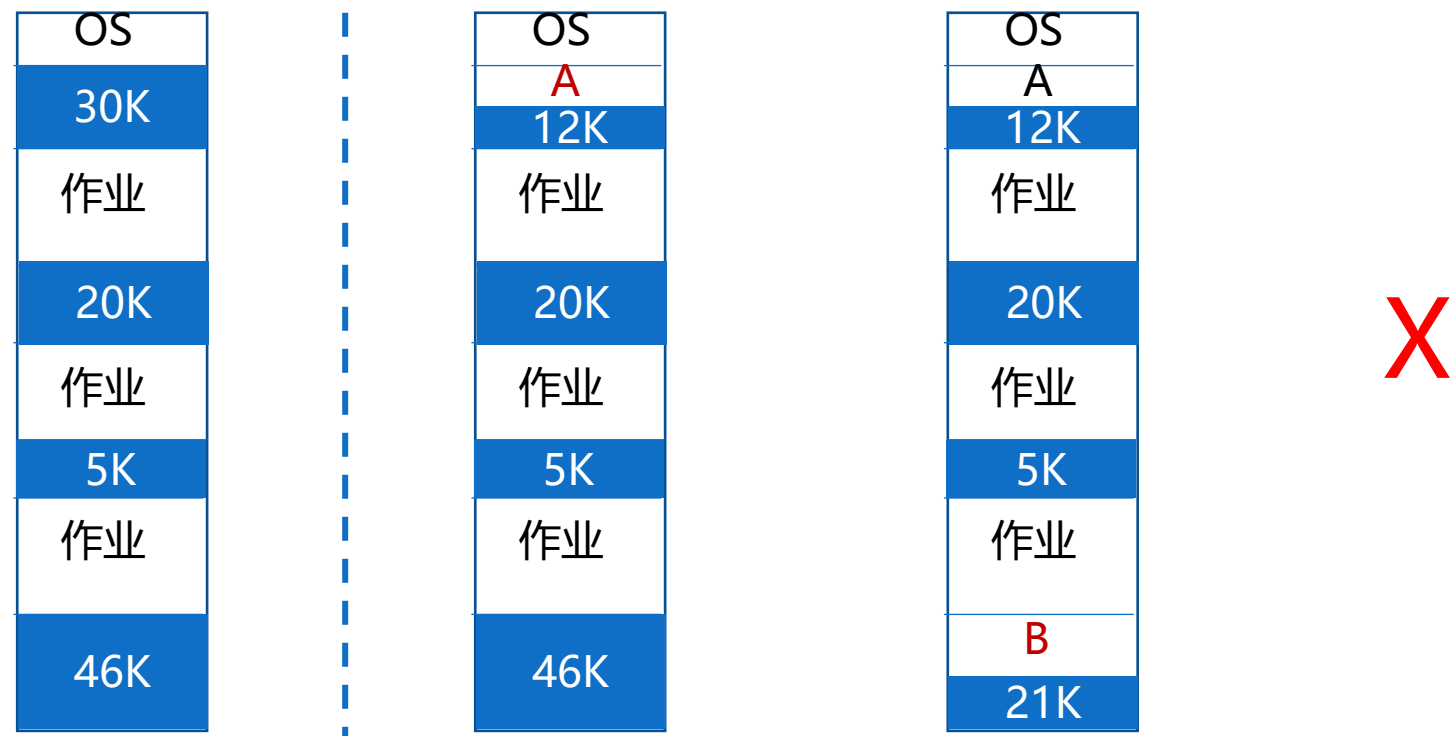
- 按分区的先后次序，找到最大的空闲分区
- 系统中出现极小空闲分片的可能性比较小，不易产生小的外碎片。但较大的空闲分区不被保留，对后继的大作业运行不利

分区分配算法

- 以上四种算法，统称为顺序搜索法
- 这些算法各有利弊，到底哪种最好不能一概而论，而应针对具体作业序列来分析。对于某一作业序列来说，某种算法能将该作业序列中所有作业安置完毕，那么该算法对这一作业序列是合适的

分区分配算法

- 例：有作业序列：作业A要求18K，作业B要求25K，作业C要求30K
- 首次/下次适配



分区分配算法

- 例：有作业序列：作业A要求18K，作业B要求25K，作业C要求30K
- 最佳适配

OS
30K
作业
20K
作业
5K
作业
46K

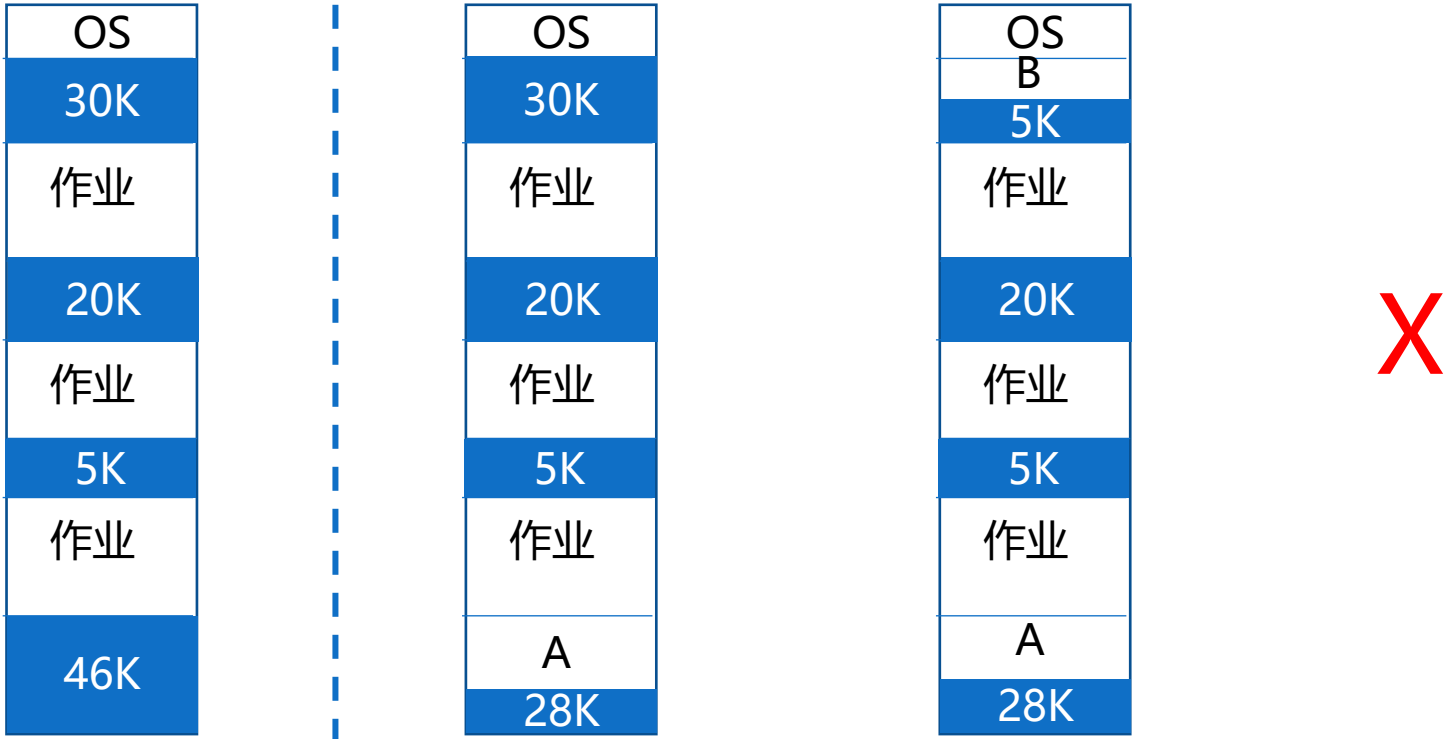
OS
30K
作业
A
2K
作业
5K
作业
46K

OS
B
5K
作业
A
2K
作业
5K
作业
46K

OS
B
5K
作业
A
2K
作业
5K
作业
C
16K

分区分配算法

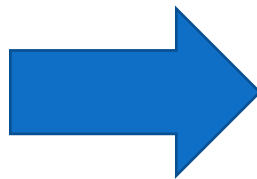
- 例：有作业序列：作业A要求18K，作业B要求25K，作业C要求30K
- 最坏适配



内存紧缩

- 动态分区分配方式中，有可能出现空闲分区总容量大于程序大小但是不能容纳该程序的情形——小空闲分区成为外碎片
- 通过程序移动将原来分散的小分区拼接成一个大分区——内存紧缩(memory compaction)
- 程序的移动需动态重定位（因程序已经装入）
- 内存紧缩十分耗时

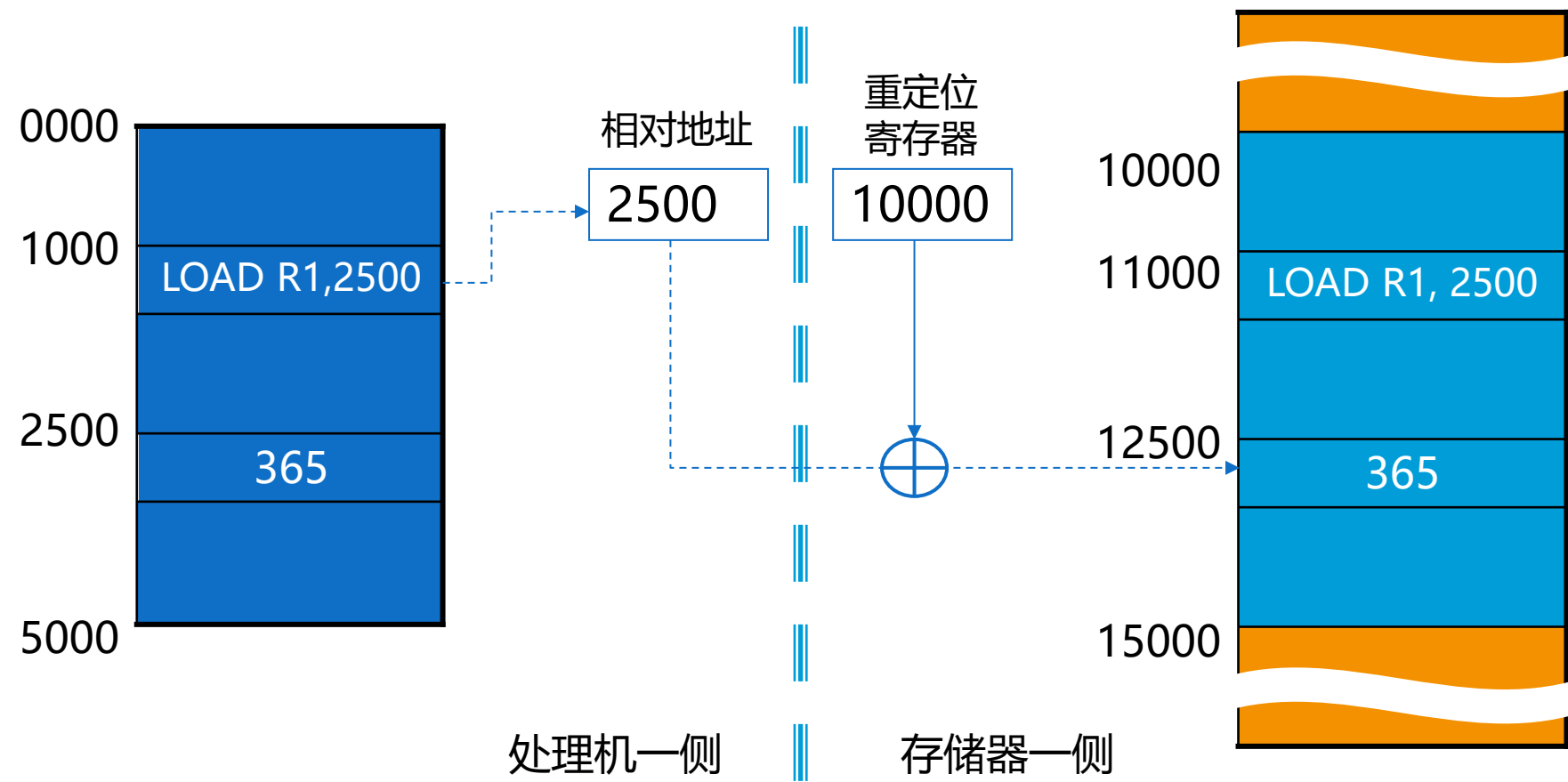
内存紧缩



分区存储管理的重定位

- 采用分区技术时，程序中使用的是相对地址，必须由硬件地址变换机构支持实现——重定位寄存器
- 重定位寄存器：存放程序在内存中的起始地址

分区存储管理的重定位



伙伴系统

- 固定分区限制了系统中活跃进程的数目，并且只能运行不超过分区大小的进程，如果进程远远小于分区大小，则内存空间的利用率非常低
- 动态分区使存储管理复杂化，并且需要系统付出紧凑外碎片的额外开销
- 伙伴系统(Buddy System)：综合固定分区技术和动态分区技术的优点

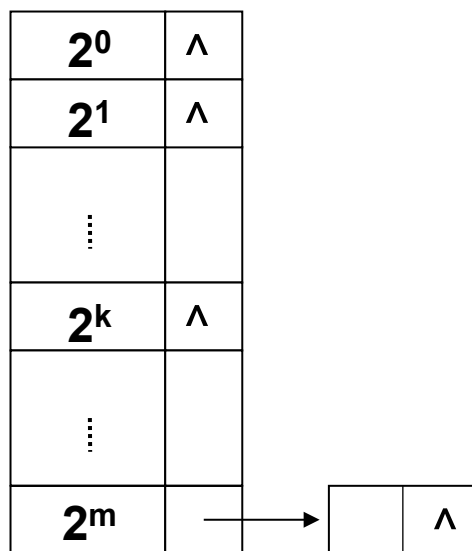
伙伴系统

- 通过不断对分大的空闲分区来获得小的空闲分区，直到获得所需要的内存分区
- 当一个空闲分区被对分后，其中的一部分称为另一部分的**伙伴(buddy)**
- 当内存分区被释放时，尽可能地合并空闲分区
- 空闲分区的分配和合并都使用2的幂次

伙伴算法

- 空闲分区和已分配分区大小都是 2^k ，其中 $0 \leq k \leq m$ ， 2^m 是整个可分配内存的大小
- 系统按不同大小的空闲分区组成 $m+1$ 个空闲分区链表

空闲分区链表的
初始状态

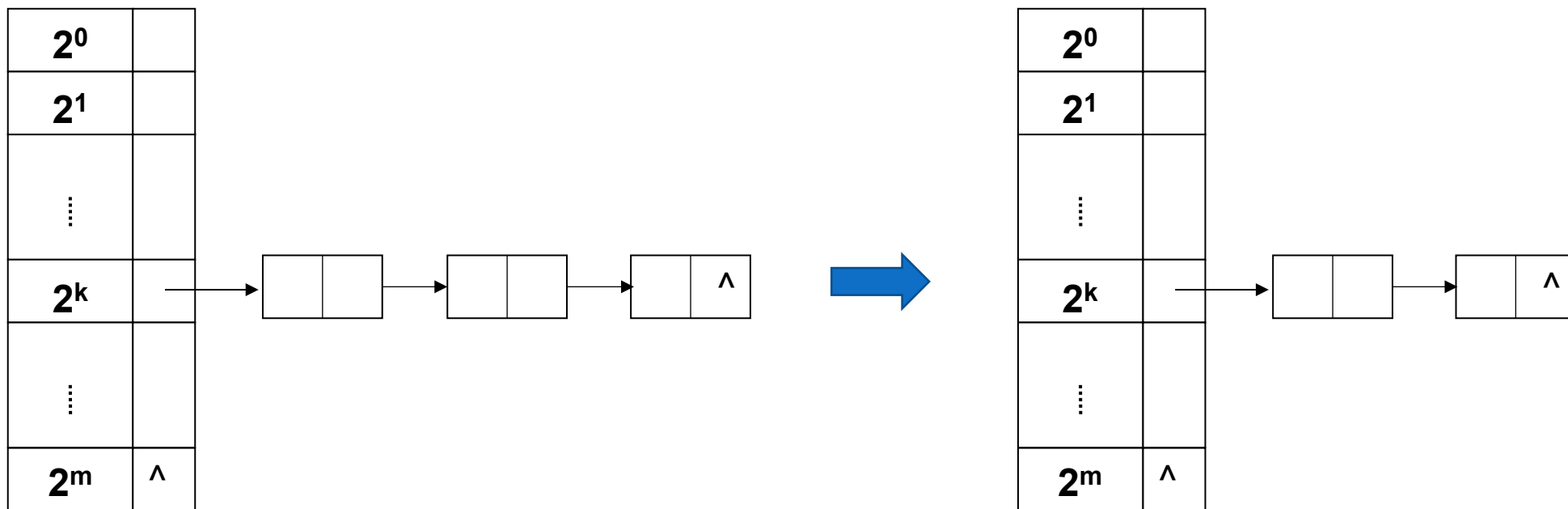


伙伴算法

- 对于大小为 n 的内存分配请求，计算 $k = \lceil \log_2 n \rceil$
- 从子链表 k 开始找空闲分区
 - 若子链表 k 非空，则将其链首结点分配之
 - 否则，若从 k 开始向下查找，直到找到 i 子链表有空闲分区($i > k$)
 - 取出 2^k 分配，剩余的分区插入相应的子链表

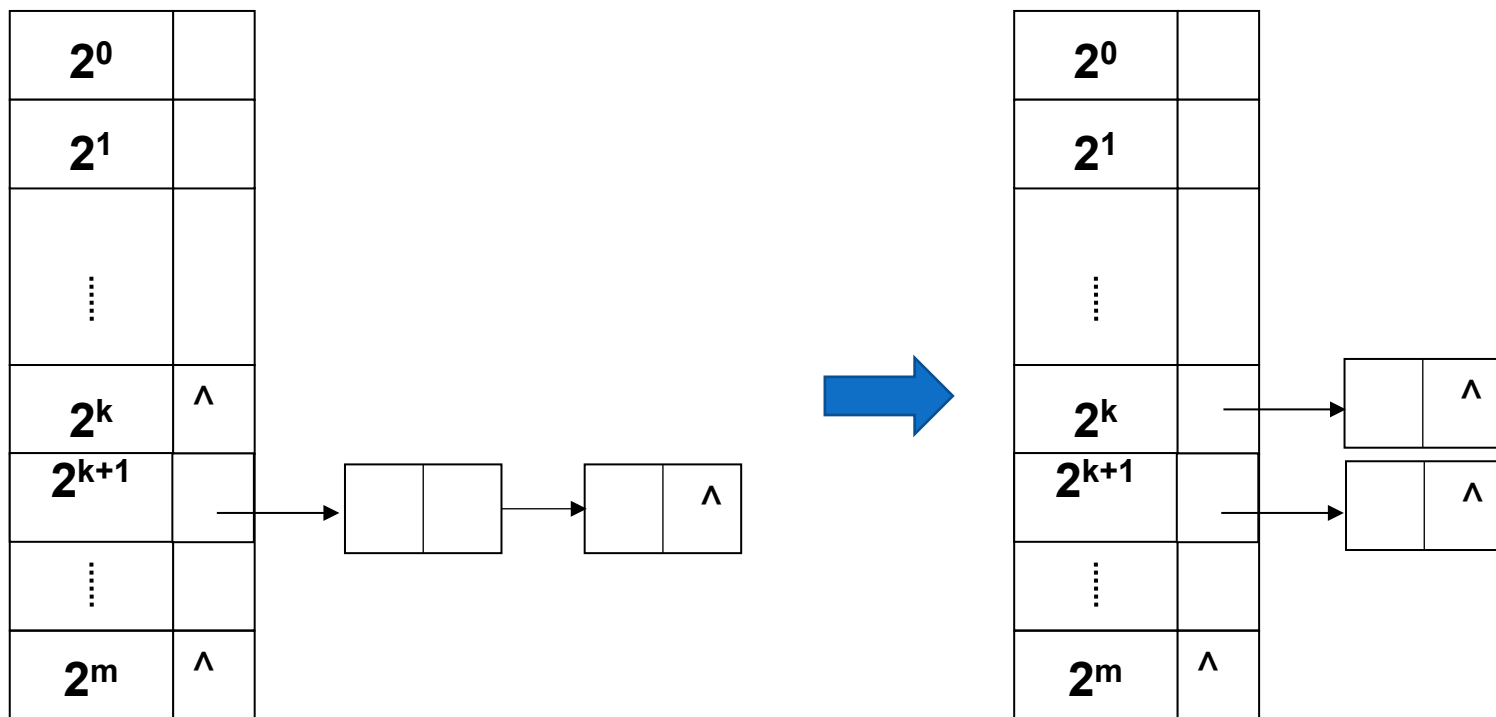
伙伴算法

- 情形1: $k = \lceil \log_2 n \rceil$, 且 k 子链表非空, 则只要删除此链表中第一个结点并分配给用户即可



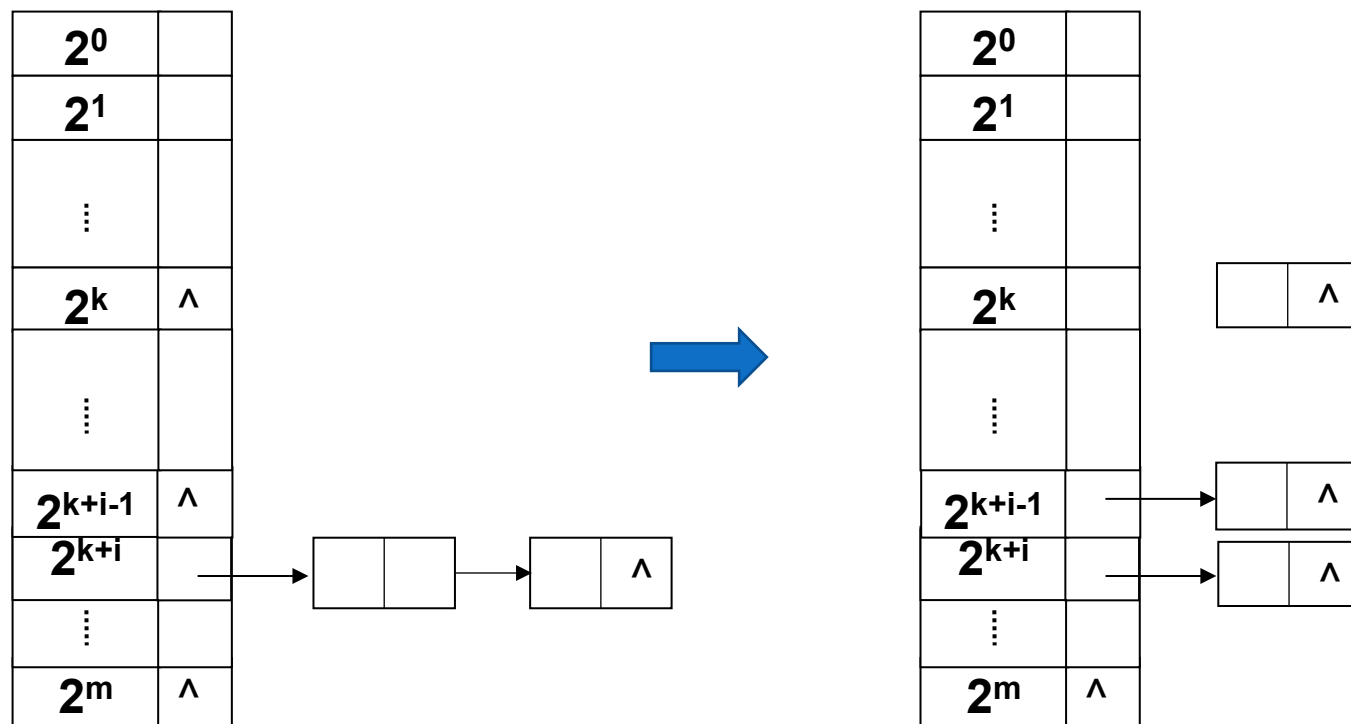
伙伴算法

- 情形2: $k = \lceil \log_2 n \rceil$, 且 k 子链表为空, 但 $k+1$ 子链表非空, 则需从 $k+1$ 子链表中取出1个结点, 将其中一半分配给用户, 剩余的一半作为一个新结点插入在 k 子链表中



伙伴算法

- 情形3: $k = \lceil \log_2 n \rceil$, 且 $k \sim k+i-1$ 子链表均为空, 但 $k+i$ 子链表非空 ($i > k$), 则需从 $k+i$ 子链表中取出1个结点, 将其中 2^k 大小的一部分分配给用户, 剩余部分分割成若干个结点分别插入在结点大小为 2^k 、 2^{k+1} 、...、 2^{k+i-1} 的子链表中



伙伴系统

系统初始状态	1M			
P1申请100KB	P1	128K	256K	512K
P2申请200KB	P1	128K	P2	512K
P3申请120KB	P1	P3	P2	512K
P4申请300KB	P1	P3	P2	P4
P2、P3结束	P1	128K	256K	P4
P5申请160KB	P1	128K	P5	P4
P1执行结束	256K		P5	P4
P5执行结束	512K			P4
P4执行结束	1M			

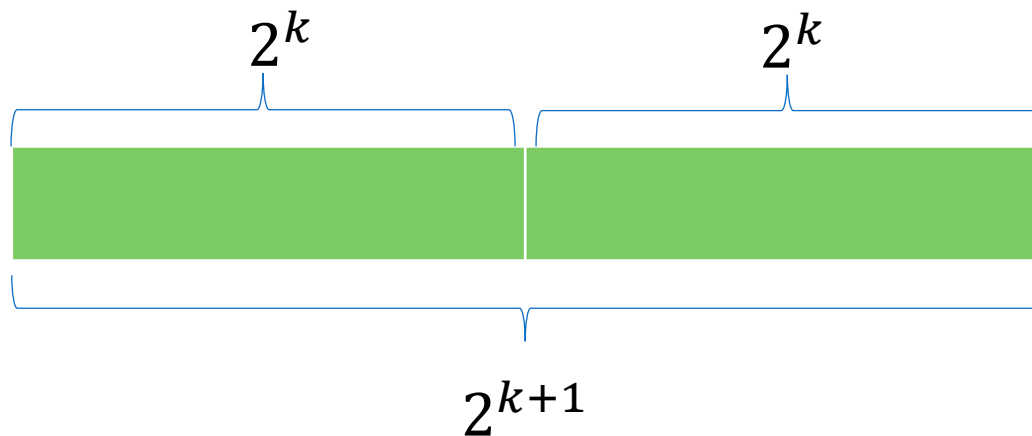
伙伴系统优缺点评价

- 多个空闲链表，可以立即找到空闲分区，算法简单、速度快
- 申请内存总是以 2^k 字节满足要求，存在内碎片。例如申请129字节，会分得256字节，申请1025字节，会分得2048字节
- 只归并伙伴而容易产生外碎片
- 申请/释放可能会导致连锁切块/合并，影响系统效率。例如，当前只有一块空闲，块大小1M，申请400字节，会导致11次切块

伙伴系统

- 在伙伴系统中，对于大小为 2^k ，地址为 x 的内存块，其伙伴块的地址为

$$byddy_k(x) = \begin{cases} x + 2^k, & \text{if } x \bmod 2^{k+1} = 0 \\ x - 2^k, & \text{if } x \bmod 2^{k+1} = 2^k \end{cases}$$



在固定分区分配中，每个分区的大小____。

- ☐ A 相同
- ☐ B 随作业长度变化
- ☒ C 可以不同但预先固定
- ☐ D 可以不同但根据作业长度固定

提交

可变分区存储管理中的内存紧缩技术，可以_____。

- ☒ A 减少外碎片
- ☐ B 减少内碎片
- ☐ C 增加内存容量
- ☐ D 加快访问速度

提交

最佳适配算法的空闲分区表_____。

- ☐ A 按大小递减顺序排列
- ☒ B 按大小递增顺序排列
- ☐ C 按地址由小到大排列
- ☐ D 按地址由大到小排列

提交

考虑一个伙伴系统，一个块地址为011011110000，如果块大小为4，它的伙伴的地址为___。

- ☒ A 011011110100
- ☐ B 011011100000
- ☐ C 011011110000
- ☐ D 111011110000

操作系统，2026

谢谢

OPERATING SYSTEM